SD242 – Verificação de Sistemas Digitais.

Autores	
Anderson Max Almeida e Porfirio	

Histórico de revisões		
05/11/2025	V1.0	Versão inicial

Página em branco.

Aula 26
Cobertura funcional

Tópicos

- Cobertura Funcional
- Tipos de Cobertura
- O que é verificado
- Metodologias de verificação
- Plano de teste da verificação
- Componentes do Plano de teste da verificação
- Coverage-Driven Verification

Cobertura Funcional

- É uma métrica de verificação que mede quão bem (percentual) o seu *conjunto de testes* exerceu a *funcionalidade* pretendida do design. Temos no design, uma combinação de todas as entradas possíveis com as combinações de estados internos.

- 100% de cobertura significa que tudo que foi indicado como métricas de cobertura foi coberto.
  - Não significa que todas as combinações que input ou transições de estados foram cobertas. Apenas as que foram pedidas para a ferramenta checar.

- Não é realista esperar 100% de cobertura funcional, mas o objetivo de um bom plano de verificação é chegar o mais próximo dos 100%.

Cobertura Funcional

- Limitações da cobertura estrutural (code coverage):
  - Apenas mostra se linhas, ramificações ou condições do código RTL foram exercutadas
  - Não garante que os requisitos funcionais foram testados
  - Pode atingir 100% de cobertura estrutural sem testar cenários importantes

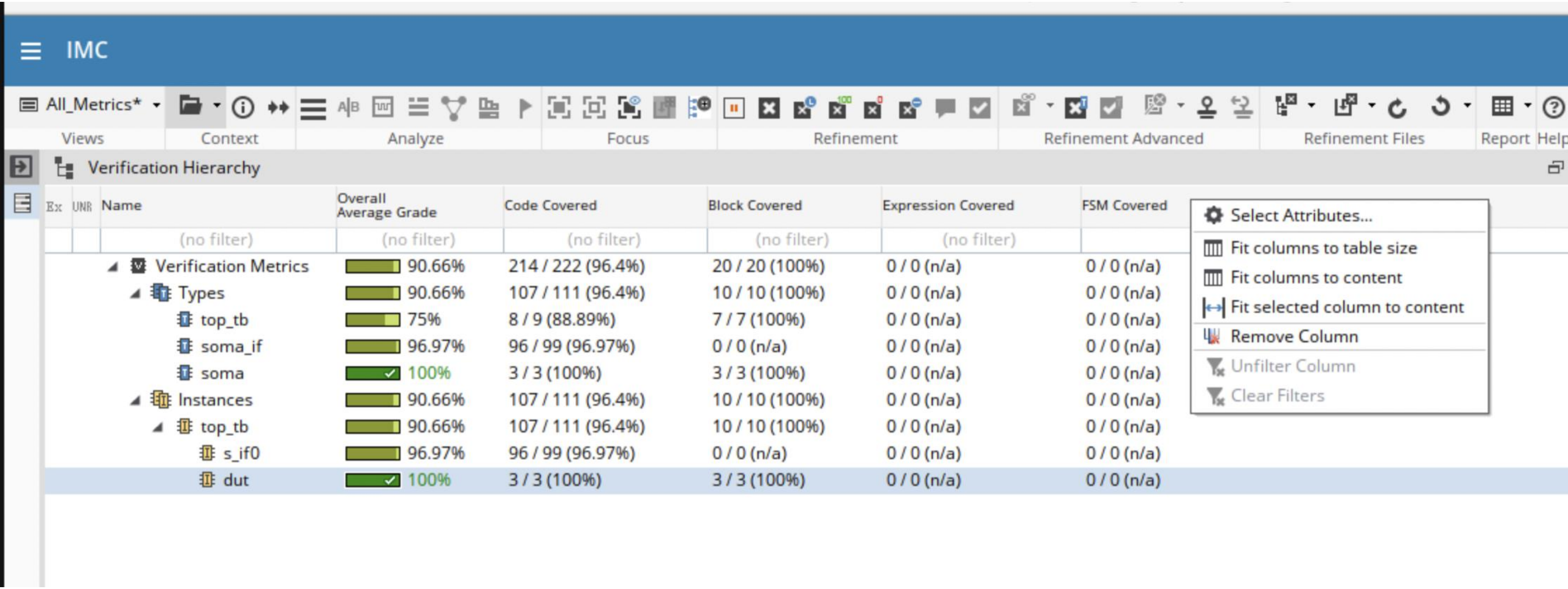
- System-on-Chip (SoCs) com múltiplos requisitos funcionais
  - Trouxe a nessidade de verificação baseada em especificações

- Verilog não tinha mecanismos nativos para cobertura funcional
  - Dependia de ferramentas proprietárias e metodologias feitas para isso

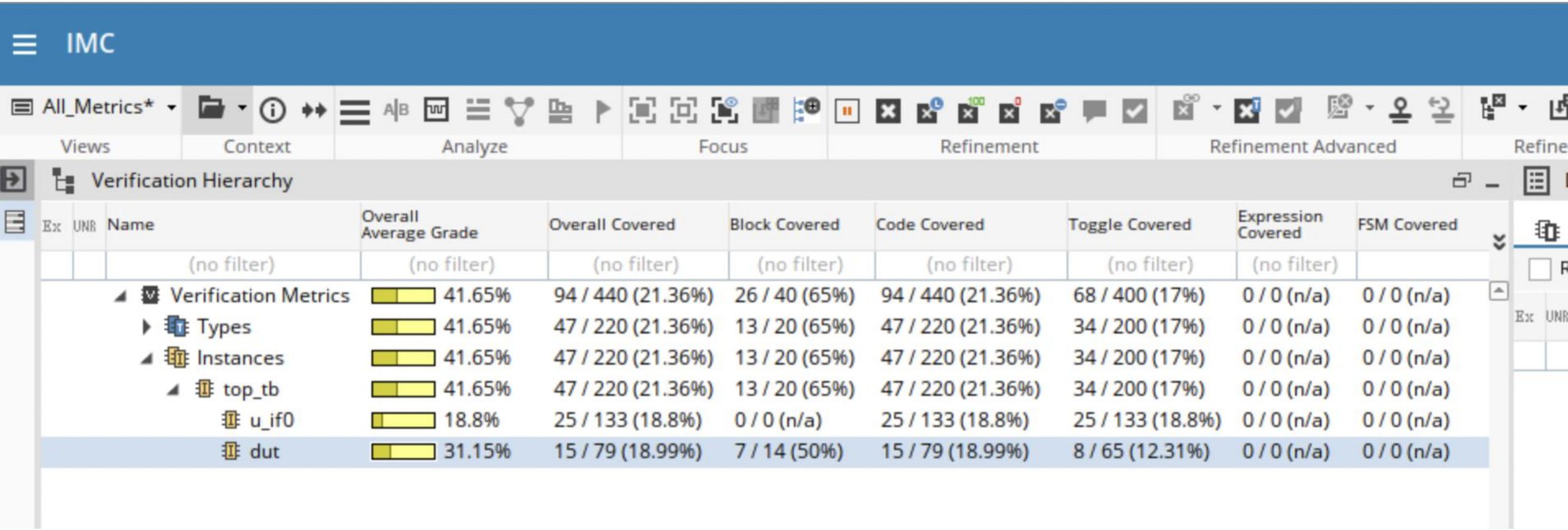
Tipos de Cobertura

- A Cobertura de Código é uma métrica **quantitativa** que indica se o código do design foi completamente estimulado pelo seu ambiente de teste.
- Cobertura de Código é baseada no código do RTL
  - Cobertura de Linha mede se cada linha executável do código RTL foi alcançada.
  - Cobertura de Bloco/Declaração faz medida de declaração (instrução) do código foi alcançada.
  - Cobertura de Condição mede se as subexpressões booleanas (em *if*, *while*, etc.) foram avaliadas para verdadeiro (True) e falso (False).
  - Cobertura de Alternância (*Toggle)* mede se cada sinal no design mudou de 0 para 1 e de 1 para 0.
  - Cobertura de FSM mede se todos os estados e transições de uma FSM foram atingidos.

Cobertura de Código



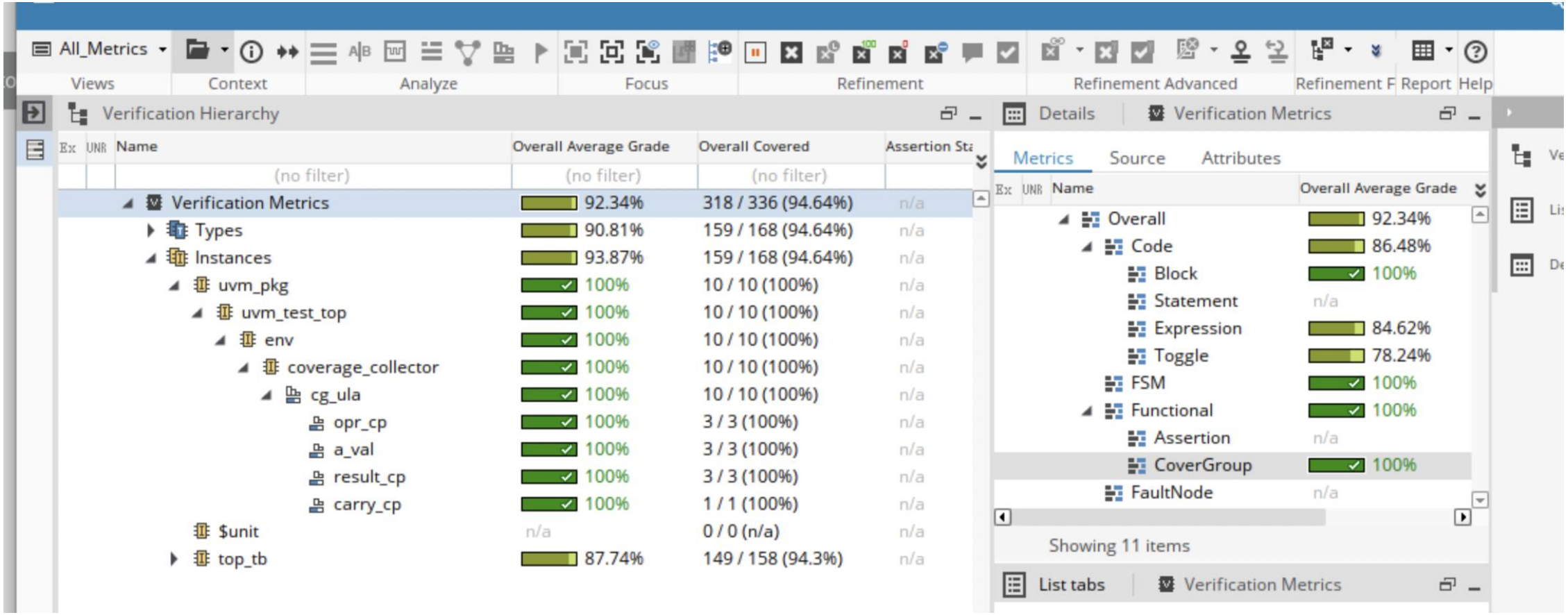
Cobertura de Código



Ex	UNR	Name	Overall Average Grade	Overall Covered	Block Covered	Code Covered	Toggle Covered	Expression Covered	FSM Covered
		(no filter)	(no filter)	(no filter)	(no filter)	(no filter)	(no filter)	(no filter)	
		Verification Metrics	41.65%	94 / 440 (21.36%)	26 / 40 (65%)	94 / 440 (21.36%)	68 / 400 (17%)	0 / 0 (n/a)	0 / 0 (n/a)
		Types	41.65%	47 / 220 (21.36%)	13 / 20 (65%)	47 / 220 (21.36%)	34 / 200 (17%)	0 / 0 (n/a)	0 / 0 (n/a)
		Instances	41.65%	47 / 220 (21.36%)	13 / 20 (65%)	47 / 220 (21.36%)	34 / 200 (17%)	0 / 0 (n/a)	0 / 0 (n/a)
		top_tb	41.65%	47 / 220 (21.36%)	13 / 20 (65%)	47 / 220 (21.36%)	34 / 200 (17%)	0 / 0 (n/a)	0 / 0 (n/a)
		u_if0	18.8%	25 / 133 (18.8%)	0 / 0 (n/a)	25 / 133 (18.8%)	25 / 133 (18.8%)	0 / 0 (n/a)	0 / 0 (n/a)
		dut	31.15%	15 / 79 (18.99%)	7 / 14 (50%)	15 / 79 (18.99%)	8 / 65 (12.31%)	0 / 0 (n/a)	0 / 0 (n/a)

Tipos de Cobertura

- Cobertura Funcional
  - System Verilog é usado para verificar um comportamento pré definido
  - A análise da cobertura é baseada no número de funções definidas que foram estimuladas.
  - Temos então um esforço manual para montar os cenários.



Coverage-Driven Verification (CDV)

- A metodologia CDV é tipicamente composta por quatro etapas principais que se repetem.
  - Planejamento da Verificação (Verification Planning)
  - Desenvolvimento do Ambiente de Verificação (Testbench)
  - Criação e Execução de Testes com Geração de Cobertura
  - Análise e Fechamento da Cobertura (Coverage Closure)

Coverage-Driven Verification (CDV)

- Análise e Fechamento da Cobertura (Coverage Closure)
  - A equipe analisa os resultados da cobertura para identificar **lacunas**. Um score de 100% é o objetivo.
  - **Ação:** Se houver lacunas (cobertura baixa), a equipe realiza uma das seguintes ações para "fechar" o gap:
    - Adicionar **Testes Direcionados (Directed Tests)** para atingir cenários específicos.
    - Ajustar as restrições da geração de testes aleatórios (constraints) para focar nas áreas não cobertas.
    - Se a lacuna for inválida (unreachable), é adicionado uma exceção na análise da cobertura.

Plano de Verificação(Verification Plan)

- Antes que um projeto possa ser verificado, um plano de verificação deve ser desenvolvido. É a especificação da verificação.
- Detalha a lista de testes necessários para validar o projeto.
- Desenvolvido a partir da especificação do projeto ASIC, que define as operações que o design deve executar.

- Os principais componentes de um plano de teste incluem
  - Escopo do plano de verificação.
  - Define o processo de verificação a ser seguido (ex.: bottom-up, top-down).
  - Especifica as ferramentas e linguagens a serem utilizadas (ex.: SV/ UVM).
  - Estabelece as métricas para avaliar quando a verificação está completa (ex.: cobertura funcional e de código).

Plano de Verificação

- Este documento fala ao time como validar o design/projeto para operar corretamente.
- Quais funções devem ser testadas.
- Como as funções serão testadas.
- Como decidir se a função está funcionando ou não.
- Documenta a arquitetura do ambiente de verificação.
- Desenvolve uma forma sistemática de garantir que toda arquitetura e componentes do projeto são testadas.

Componentes do Plano de Verificação

- Definir o ambiente de verificação
  - Descrever informações importantes da verificação
  - Criar diagramas de bloco do ambiente
  - Tabela de versão, autor, revisões, projeto etc.
  - Metodologia de verificação
    - Configuração do TB
    - Verificação formal
    - Verificação com assertions
  - Linguagem utilizada
  - Quaisquer outras informações relevantes sobre o ambiente

Componentes do Plano de Verificação

- Lista todas as funcionalidades críticas do projeto que devem ser verificadas.
- Cada funcionalidade deve incluir uma descrição do comportamento funcional ou de timing esperado em alto nível.
  - "A máquina de estado deve transitar por todos os estados válidos."
  - "Todos os endereços válidos na SRAM são graváveis."
  - "Todos os endereços válidos na SRAM são legíveis em dois ciclos."
- A lista é composta por
  - Funcionalidades do projeto que devem ser testadas
  - Requisitos funcionais/de temporização que devem ser testados
- A lista geralmente é desenvolvida com o arquiteto do projeto.
- A lista deve ser priorizada, separando funcionalidades que devem ser testadas daquelas que são desejáveis de serem testadas.

Componentes do Plano de Verificação

- A lista de testes deve conter uma lista completa de todos os testes que devem ser executados para verificar as funcionalidades listadas na lista de funcionalidades.
  - Para cada teste descrito na lista de testes, documente o seguinte:
  - Descrição detalhada do teste
  - Método de teste (simulação, verificação formal ou ABV)
  - Nível de teste (unidade, grupo/transacional, sistema)
  - Funcionalidade(s) que o teste verificará (use os números da funcionalidade da lista de funcionalidades)
  - Prioridade do teste
  - Proprietário do teste (pessoa responsável por desenvolver)

Componentes do Plano de Verificação

- Requisitos de Cobertura e Conclusão
- Os critérios de conclusão podem incluir:
  - Métricas de cobertura
    - Os testes cessam quando os níveis de cobertura e funcional atingem um nível pré-determinado.
    - Coberturas estruturais também devem ser avaliadas e analisadas.
  - Métricas de bugs
    - Padrões de detecção de bugs que ocorrem durante a simulação.
    - Conforme cada teste de unidade ou integração começa, a taxa de detecção de bugs tende a ser relativamente alta, diminuindo à medida que o teste da subseção continua.
    - Quando a taxa de detecção de bugs cai abaixo de um nível pré-determinado para uma parte do sistema, então temos um bom grau de confiança no projeto do bloco.

Componentes do Plano de Verificação

- Para cada a verificação, o seguinte deve ser estimado:
  - Recursos de engenharia necessários
  - Nível de experiência dos engenheiros necessários
  - Requisitos de recursos de computador
  - Requisitos de licença de software

Engineering Resources	qtd	descriçao	data	duração
Senior Verification Engineer	1	10 - 12 years experience		6 meses
Verification Engineer	2	2-5 years experience		6 meses
Junior Verification Engineer	1	1 year experience		6 meses
Compute Resources				
System Verif Server	1	Dual Core Athlon CPU, 32 GB RAM		6 meses
Unit Verif Server	2	Single core Athlon CPU, 8 GB RAM		6 meses
Software/License Resources				6 meses
Cadence Incisive	3	Full seat of Incive XL		6 meses
Cadence Encounter Conformal	1	Conformal ASIC license		3 meses

Componentes do Plano de Teste da Verificação

- Cronograma (Schedule)
  - Crie um cronograma mestre que acompanhe qualquer recurso que possa causar a paralisação do esforço de verificação, caso não seja gerenciado adequadamente.
  - Exemplos:
  - Recurso(pessoas) - Início - Duração - Ação/Tarefa - Recurso de Computação - Licenças de Software - Engenheiro Responsável

Coverage-Driven Verification (CDV)

- A metodologia CDV é tipicamente composta por quatro etapas principais que se repetem.
  - Planejamento da Verificação (Verification Planning)
  - Desenvolvimento do Ambiente de Verificação (Testbench)
  - Criação e Execução de Testes com Geração de Cobertura
  - Análise e Fechamento da Cobertura (Coverage Closure)

Functional Coverage

- Os aspectos-chave da cobertura funcional são os seguintes:
  - É especificada pelo usuário e não é inferida automaticamente do projeto.
  - É baseada na especificação do projeto (ou seja, sua intenção).
  - Alguém tem que escrever o modelo de cobertura.
  - SystemVerilog tem uma sintaxe na linguagem para uma fácil especificação do modelos de cobertura funcional.
  - A própria simulação do SV gera os arquivos que podem ser analisados por ferramentas de análise de cobertura funcional.

Functional Coverage

- Covergroup
  - É um construct que encapsula um modelo de cobertura, funcionando como uma "classe de cobertura" que pode ser instanciada múltiplas vezes.
  - Ele define o que será medido, quando será medido e como os dados serão organizados.

- Coverage Points (Pontos de Cobertura)
  - São elementos dentro de um covergroup que especificam o que será medido, geralmente uma variável ou expressão (integral ou real).
  - Rastrea valores ou transições de valores que ocorrem durante a simulação.

- Cobertura Cruzada (Cross Coverage)
  - Mede a cobertura combinada de dois ou mais pontos de cobertura ou variáveis.
  - Verifica se combinações específicas de valores de diferentes pontos de cobertura foram exercitadas (ex: comando cmd com endereço addr).

Laboratório1 - coverage

- Crie o Verification Plan para um DUT (módulo/bloco/IP) que será desenvolvido por você no Trabalho Orientado II deste curso, ou outro módulo qualquer de sua escolha.
  - Ao mapear as funcionalidades na especificação, coloque tags enumerando-as.
  - Ao fazer a lista de testes, use as tags que o teste cobre.
- Envie no moodle a especificação do módulo junto com o plano de verificação.

Functional Coverage

- Os aspectos-chave da cobertura funcional são os seguintes:
  - É especificada pelo usuário e não é inferida automaticamente do projeto.
  - É baseada na especificação do projeto (ou seja, sua intenção).
  - Alguém tem que escrever o modelo de cobertura.
  - SystemVerilog tem uma sintaxe na linguagem para uma fácil especificação do modelos de cobertura funcional.
  - A própria simulação do SV gera os arquivos que podem ser analisados por ferramentas de análise de cobertura funcional.

Functional Coverage

- As construções de cobertura funcional do SystemVerilog permitem o seguinte:
  - Cobertura de variáveis e expressões, bem como cobertura cruzada entre elas
  - Bins de cobertura automáticos e definidos pelo usuário
  - Associar bins a conjuntos de valores, transições ou produtos cruzados
  - Condições de filtragem em múltiplos níveis
  - Eventos e sequências para acionar automaticamente a amostragem de cobertura
  - Ativação e consulta procedural da cobertura
  - Diretivas opcionais para controlar e regular a cobertura

Covergroup

- É um tipo definido pelo usuário.
- A definição do tipo é escrita uma vez, e múltiplas instâncias desse tipo podem ser criadas em diferentes contextos.
- Uma instância de covergroup pode ser criada através do operador new().
- Um covergroup pode ser definido em um pacote, módulo, interface ou classe.

```
covergroup cg; ... endgroup
cg cg_inst = new;
```

- Em um covergroup podemos definir argumentos.
- A especificação extends de covergroup é permitida apenas dentro de uma classe.

Covergroup

- O coverage_event define (opcional) o evento no qual os pontos de cobertura são amostrados.
  - Evento de clocking
  - Expressão de evento de bloco
  - Método sample()

```
enum {red, green, blue} color;

covergroup g1 @(posedge clk);
   c: coverpoint color;
endgroup
```

- Como o coverpoint não define explicitamente nenhum bin, a ferramenta cria automaticamente três bins, um para cada valor possível do tipo enumerado
  - Se o número que cores (bins) do design aumentar, precisamos acrescentar no enum e aumentar o número de randomização dos estímulos.

Covergroup / coverpoint

```
module test(); // (input logic clk)
    bit[1:0] a, b;
    covergroup cg; // @(posedge clk)
        coverpoint a;
        coverpoint b;
    Endgroup
    cg cg0;            // dentro de uma classe não precisa criar a referencia do modulo. (Código)
    initial begin
        cg0 = new():
        repeat (4 ) begin
            a = $random();
            b = $random();
            cg0.sample(); // se sample no posedge, não precisa usar sample()
        end
    end
endmodule
```

Herança

- O covergroup filho herda todos os coverpoints, crosses e options definidos no covergroup pai.
- No filho, pode adicionar novos coverpoints e crosses.
- Modificar os bins de um coverpoint herdado, usando a sintaxe super.coverpoint_name para manter os bins do pai ou o nome do coverpoint herdado para substituí-los.
- Sobrescrever opções (como option.name ou option.comment).
- A principal utilidade de estender covergroups é promover a reutilização e a escalabilidade do ambiente de verificação, alinhando-se aos princípios da UVM

Covergroup - Opções

- Option.at_least
  - Define o número mínimo de vezes que um bin deve ser atingido (hit) para ser considerado coberto.
  - option.at_least = 2; (para cg ou para cp)
- Option.goal
  - Define a pontuação alvo (em porcentagem) do covergroup para que ele seja considerado completo.
  - option.goal = 90;
- Option.comment
  - Adiciona uma string de comentário que é incluída no banco de dados de cobertura, ajudando na documentação.
  - option.comment = "Cobertura de Registros Críticos";

Covergroup - Opções

- type_option.weight
  - Define o peso que o covergroup terá no cálculo do score final de toda a verificação.
  - type_option.weight = 5;
- option.per_instance
  - Controla se a cobertura deve ser coletada por instância ou se a cobertura de todas as instâncias deve ser mesclada.
  - option.per_instance = 1;
- cross_option.at_least
  - Semelhante ao option.at_least, mas se aplica apenas aos bins de cobertura cruzada (combinações).
  - cross_option.at_least = 5;

Coverpoint - Bins

- illegal_bins
  - illegal_bins forbidden_value = {8'h7F};
  - Define um conjunto de valores (ou faixas) para um coverpoint que, se for amostrado (sample()), indica uma falha grave na verificação.
  - Ação da Ferramenta: Quando um illegal_bin é atingido, o simulador registra um erro fatal.
  - Garante que os testes não estejam gerando estímulos inválidos ou que o Design Under Test (DUT) não esteja entrando em um estado proibido.
- ignore_bins
  - ignore_bins middle_range = {[100:155]};
  - Define um conjunto de valores que, embora possam ser válidos e amostrados, não devem contribuir para o cálculo do score de cobertura final.
  - Elimina a necessidade de testar exaustivamente áreas do design que se sabe serem trivialmente corretas.

Coverpoint - Bins

- Bins Default
  - Ao não definir explicitamente todos os bins possíveis (como foi feito para a), os valores restantes são implicitamente incluídos em um bin default.
- Bins de Transição (carry_o)
  - bins carry_toggle = (0 => 1), (1 => 0);
  - Mede a Cobertura de Transição. Garante que o sinal carry_o não apenas atinja os valores 0 e 1, mas também que ele faça a transição de 0 para 1 e depois de 1 para 0.
- ignore_bins em cross
  - Permite que você defina quais combinações específicas de bins você não precisa cobrir.
  - Exemplo: Se a combinação de duas entradas baixas (low_x_low) for sempre trivial, você a ignora para focar os recursos de simulação em combinações mais complexas.

Coverage Closure

- Fechamento da Verificação
  - A verificação termina quando:

    A cobertura atinge um score pré-definido.

  - Até lá devemos:

    Identificar as lacunas (holes) de cobertura (os bins não atingidos).

    Gerar novos Testes Direcionados ou refinar as Restrições de Randomização (CRV) para preencher as lacunas.

    Adicionar exceções para bins que são comprovadamente inalcançáveis.

Laboratório

- Use Cobertura Funcional para verificar se a ULA simples foi testada em todos os seus modos de operação e com valores de dados significativos.
  - No RTL da DUT encontramos as funcionalidades que devem ser testadas.
  - Desenvolva/adapte os testes que irão complementar a cobertura funcional.
  - Desenvolva as lógicas de coletar cobertura funcional para a funcionalidades que ainda não foram desenvolvidas.
  - Envie os arquivos em uma pasta zip no moodle.

Referências

- IEEE Standard for SystemVerilog (Cap 19)

Página em branco.